

# A NUMERICAL METHOD FOR SOLVING THE EIKONAL EQUATION ON SIMPLICIAL TESSELATIONS\*

SACHIN NATESH<sup>†</sup>

**Abstract.** In this paper, we describe a method for solving the Eikonal equation on domains in  $\mathbb{R}^2$  or  $\mathbb{R}^3$  tessellated by triangles or tetrahedra. Our method generalizes to simplices in higher dimensions, owing to our choice of solution representation under the Jacobi family of orthogonal polynomials on the simplex. First, we describe a spectral decomposition for functions supported on the simplex in terms of their weighted  $L_2$  expansion coefficients under the Jacobi basis. Computing this expansion requires an accurate and efficient quadrature, and we provide a novel optimization routine based on approximate joint-diagonalization of Jacobi matrices for generating such quadratures. Then, we consider differential operators acting on coefficients of a Jacobi expansion. Recent results due to Townsend, Oliver and Vasil enable the construction of sparse differential operators on the standard triangle, interoperating within the Jacobi polynomial family parameter space, and leading to low-storage, banded discrete differential operators. We generalize these results to the tetrahedra (laying the path for generalizing to  $d > 3$ ), and use the resulting differential operators to construct the Eikonal operator on the simplex. Ultimately, we must solve a non-linear PDE, and working in coefficient space implies that our solution variables must be thought of as functions. Hence, we reformulate the Eikonal problem in terms of an infinite-dimensional  $L_2$  minimization problem, which can be discretized with the aforementioned numerical quadrature. As with the README, this is more of a blog, but it's much easier to typeset math here.

**Key words.** multivariate orthogonal polynomials, spectral methods, partial differential equations, Eikonal equations, simplex

**MSC codes.** 68Q25, 68R10, 68U05

## 1. Global representations to local solutions of the Eikonal Equation.

Haven't really looked into the viscosity perspective, but..I think I converged on the same idea for solving the PDE. Consider the standard simplex

$$\mathcal{T}^d = \{\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d \mid x_i \geq 0, 1 - |\mathbf{x}|_1 \geq 0\}$$

The *minimal* time  $u(\mathbf{x})$  to reach the boundary  $\partial\mathcal{T}^d$  from a point  $\mathbf{x} \in \mathcal{T}^d$  is computable through solving the Eikonal equation on  $\mathcal{T}^d$ :

$$(1.1) \quad \begin{cases} \sum_{i=1}^d (\partial_{x_i} u)^2 = \left(\frac{1}{f}\right)^2, & \mathbf{x} \in \mathcal{T}^d, \\ u = q, & \mathbf{x} \in \partial\mathcal{T}^d, \end{cases}$$

where  $f : \overline{\mathcal{T}^d} \rightarrow \mathbb{R}_*^+$  is the speed in the medium,  $q : \partial\mathcal{T}^d \rightarrow \mathbb{R}^+$  is the time to leave  $\mathcal{T}^d$  from a point on  $\partial\mathcal{T}^d$ . We seek to represent  $u$  under an optimal, finite, weighted  $L_2$  expansion, wherein

$$u(\mathbf{x}) \approx \sum_{j=1}^n u_j P_j(\mathbf{x}) = \mathbf{u} \cdot \mathbf{P}(\mathbf{x}),$$

and  $\mathbf{u}$  is the vector in  $\mathbb{R}^n$  of coefficients of  $u$  under the elements of the basis  $\mathbf{P}(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^n$ . Here  $\approx$  must be taken in the sense of weighted  $L_2$  convergence of the series to  $u$  as  $n \rightarrow \infty$ . For the typical case of  $q \equiv 0$  (i.e. time to boundary is 0 if on the boundary), we employ a weighted basis which evaluates to 0 on the boundary, thereby naturally satisfying homogeneous Dirichlet conditions.

---

\*Submitted to the editors DATE.

**Funding:** This work is partially supported by the National Science Foundation under Grant Number 2110886.

<sup>†</sup>Department of Applied Mathematics, University of Colorado, Boulder, CO

**1.1. Jacobi bases on the Triangle.** Let us specialize for  $d = 2$ . By  $P_k^n$ , we denote the  $k^{\text{th}}$  Jacobi polynomial on the triangle of degree  $n$  with parameters  $(a, b, c)$ . When the parameters must be specified, we will refer to the same polynomial by  $P_{k,n}^{(a,b,c)}$ . The weight function is

$$(1.2) \quad \begin{aligned} W^{(a,b,c)}(x, y) &= x^{a-\frac{1}{2}} y^{b-\frac{1}{2}} (1-x-y)^{c-\frac{1}{2}}, \\ w_{(a,b,c)} &= \left( \int_{\mathcal{T}_{\text{ref}}^2} W(x, y) dx dy \right)^{-1} = \frac{\Gamma(|\kappa| + \frac{3}{2})}{\Gamma(a + \frac{1}{2}) \Gamma(b + \frac{1}{2}) \Gamma(c + \frac{1}{2})} \\ w^{(a,b,c)}(\mathbf{x}) &= w_{(a,b,c)} W^{(a,b,c)}(\mathbf{x}) \end{aligned}$$

with  $a, b, c > -1/2$  the Jacobi parameters, and  $|\kappa| = a + b + c$ . The orthonormal polynomials are given in terms of the 1D Jacobi polynomials by

$$(1.3) \quad \begin{aligned} P_k^n(x, y) &= (h_{k,n})^{-1} P_{n-k}^{2k+b+c, a-1/2} (2x-1) (1-x)^k P_k^{c-1/2, b-1/2} \left( \frac{2y}{1-x} - 1 \right), \\ (h_{k,n})^2 &= \frac{w_{a,b,c}}{(2n + |\kappa| + \frac{1}{2})(2k + b + c)} \\ &\quad \times \frac{\Gamma(n + k + b + c + 1) \Gamma(n - k + a + \frac{1}{2}) \Gamma(k + b + \frac{1}{2}) \Gamma(k + c + \frac{1}{2})}{(n - k)! k! \Gamma(n + k + |\kappa| + \frac{1}{2}) \Gamma(k + b + c)}, \end{aligned}$$

and they satisfy a mutual orthonormality condition under the weighted inner product:

$$(1.4) \quad \langle P_{j,m}^{(a,b,c)}, P_{k,n}^{(a,b,c)} \rangle_{w^{(a,b,c)}} = \delta_{jk} \delta_{mn}.$$

A given basis polynomial is not only orthonormal to those with different total degree, but also to those others within the same-degree homogeneous subspace. In other words,  $P_k^n$ ,  $0 \leq k \leq n$  forms a basis for the space of homogeneous polynomials of degree  $n$ . By considering ordered collections of these subspaces (where ordering is in terms of the degree  $n$  of the space), instead of individual polynomials, we can more elegantly reformulate the Eikonal problem into a weak-type form. Let

$$\mathbb{P}_n(x_1, x_2) = (P_0^n(x_1, x_2), \dots, P_n^n(x_1, x_2))^T,$$

and let

$$(1.5) \quad \mathbf{P} = (\mathbb{P}_0^T, \mathbb{P}_1^T, \dots, \mathbb{P}_{n-1}^T)^T.$$

The above notation amounts to a vectorization of the basis for the general space of polynomials in two variables  $\Pi_{n-1}^2$  defined of  $\mathcal{T}^2$ . Now, consider the  $(n-1)$  order Jacobi polynomial expansion of the solution  $u$  under the parameter family  $(a, b, c)$ . For  $\mathbf{x} \in \mathcal{T}^d$ , and  $N = \dim \Pi_{n-1}^2$  coefficients  $\mathbf{u} \in \mathbb{R}^N$ , we can write it as

$$(1.6) \quad u(\mathbf{x}) = \mathbf{P}^{(a,b,c)}(\mathbf{x})^T \mathbf{u}.$$

There exists sparse discrete operators which map  $\mathbf{u}$  to the coefficients of the derivative of  $u$  under a modified basis, as well as those which promote the basis parameters without taking derivatives:

$$(1.7) \quad \begin{aligned} \partial_{x_1} u(\mathbf{x}) &= \mathbf{P}^{(a+1, b+1, c+1)}(\mathbf{x})^T \mathcal{K}_{(a+1, b, c+1)}^{(a+1, b+1, c+1)} \mathcal{D}_{x_1} \mathbf{u} \\ \partial_{x_2} u(\mathbf{y}) &= \mathbf{P}^{(a+1, b+1, c+1)}(\mathbf{x})^T \mathcal{K}_{(a, b+1, c+1)}^{(a+1, b+1, c+1)} \mathcal{D}_{x_2} \mathbf{u} \end{aligned}$$

Inserting this into the fully constrained Eikonal equation gives

$$\begin{aligned}
 (1.8) \quad & \left( \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x})^T \mathcal{K}_{(a,b,c)}^{(a+1,b+1,c+1)} \mathbf{c}_{\frac{1}{f}} \right)^2 = \\
 & \left( \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x})^T \mathcal{K}_{(a+1,b,c+1)}^{(a+1,b+1,c+1)} \mathcal{D}_{x_1} \mathbf{u} \right)^2 + \\
 & \left( \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x})^T \mathcal{K}_{(a,b+1,c+1)}^{(a+1,b+1,c+1)} \mathcal{D}_{x_2} \mathbf{u} \right)^2 = \\
 (1.9) \quad & \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x})^T (\tilde{\mathcal{D}}_{x_1} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_2}^T) \mathbf{P}^{(a+1,b,c+1)}(\mathbf{x}),
 \end{aligned}$$

where

$$(1.10) \quad \tilde{\mathcal{D}}_{x_1} = \mathcal{K}_{(a+1,b,c+1)}^{(a+1,b+1,c+1)} \mathcal{D}_{x_1}, \quad \tilde{\mathcal{D}}_{x_2} = \mathcal{K}_{(a,b+1,c+1)}^{(a+1,b+1,c+1)} \mathcal{D}_{x_2},$$

and, the coefficients of  $(1/f)$  are represented as  $\mathbf{c}_{\frac{1}{f}}$  but promoted to the  $(a+1, b+1, c+1)$  Jacobi basis. We let  $\tilde{\cdot}$  indicate promotion *to*  $(a+1, b+1, c+1)$  from any other parameter set. The above equality holds for any point  $\mathbf{x} \in \mathcal{T}^2$ . So, for a solution  $\mathbf{u}$ , the coefficient operation must be such that:

$$(1.11) \quad (\tilde{\mathcal{D}}_{x_1} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_2}^T) = \tilde{\mathbf{c}}_{\frac{1}{f}} \tilde{\mathbf{c}}_{\frac{1}{f}}^T.$$

We seek a coefficient solution  $\mathbf{u}$  which represents the solution *globally* in  $\mathcal{T}^2$ . However, polynomials are differentiable, and some of the problems we are interested in yield non-differentiable solutions. This is true even in the most basic case of  $f \equiv 1$ , in which  $u$  is the signed distance function from  $\mathbf{x}$  to  $\partial\mathcal{T}^2$ , depicted below:

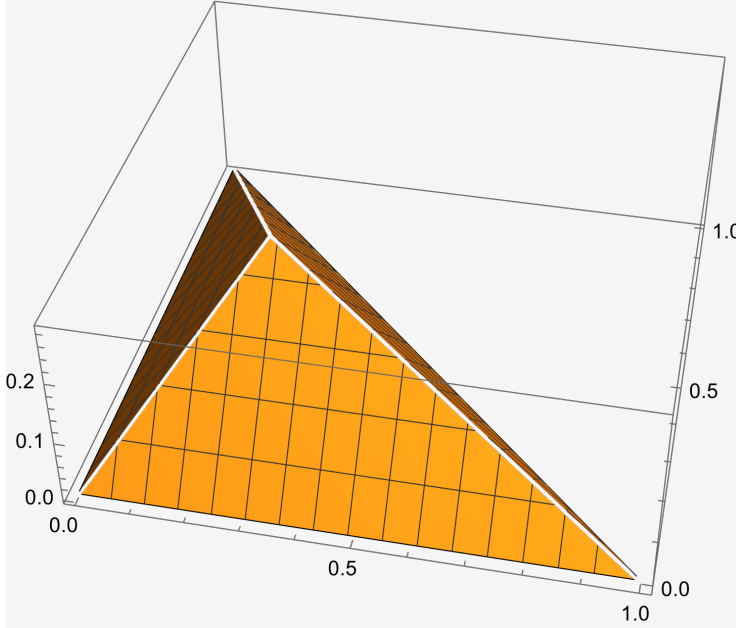


FIG. 1. A simple calculation shows that the minimum distance from  $\mathbf{x} \in \mathcal{T}^2$  to the boundary  $\partial\mathcal{T}^2$  is given by the minimum of  $\left\{ x_1, x_2, \left\| \mathbf{x} - \begin{bmatrix} (x_1 - x_2 + 1)/2 \\ (x_2 - x_1 + 1)/2 \end{bmatrix} \right\|_2 \right\}$

We cannot hope to recover a solution  $\mathbf{u}$  which satisfies (1.11) to machine precision, as that would imply the global solution has zero residual *pointwise* everywhere. Hence,

we must return to (1.8), and consider solving it weakly, in the sense of:

$$\begin{aligned}
(1.12) \quad & \int_{\mathcal{T}^2} w^{(a+1,b+1,c+1)}(\mathbf{x}) \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x})^T \tilde{\mathbf{c}}_{\frac{1}{f}} \tilde{\mathbf{c}}_{\frac{1}{f}}^T \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x}) d\mathbf{x} \approx \|1/f\|_{w^{(a+1,b+1,c+1)}}^2 \\
& \approx \int_{\mathcal{T}^2} w^{(a+1,b+1,c+1)}(\mathbf{x}) \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x})^T (\tilde{\mathcal{D}}_{x_1} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_2}^T) \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x}) d\mathbf{x} \\
& \approx \|\partial_x u\|_{w^{(a+1,b+1,c+1)}}^2 + \|\partial_y u\|_{w^{(a+1,b+1,c+1)}}^2. \quad \blacksquare
\end{aligned}$$

Here, the norm is viewed as that of a function in  $L^2(\mathcal{T}^2, w^{(a+1,b+1,c+1)})$ . Parsing this equation out a bit further (condensing sub/super scripts), we have

$$\begin{aligned}
& \int_{\mathcal{T}^2} \mathbf{P}(\mathbf{x})^T (\mathbf{u}_x \mathbf{u}_x^T + \mathbf{u}_y \mathbf{u}_y^T) w(\mathbf{x}) \mathbf{P}(\mathbf{x}) d\mathbf{x} \quad (\text{Only } x \text{ part handled below}) \\
& = \int_{\mathcal{T}^2} \mathbf{P}(\mathbf{x})^T \begin{bmatrix} u_{x,1}^2 & u_{x,1}u_{x,2} & \cdots & u_{x,1}u_{x,N} \\ u_{x,2}u_{x,1} & u_{x,2}^2 & \cdots & u_{x,2}u_{x,N} \\ \vdots & \ddots & \ddots & \vdots \\ u_{x,N}u_{x,1} & u_{x,N}u_{x,2} & \cdots & u_{x,N}^2 \end{bmatrix} \mathbf{P}(\mathbf{x}) w(\mathbf{x}) d\mathbf{x} \\
& = \int_{\mathcal{T}^2} \begin{bmatrix} P_1(\mathbf{x}) & P_2(\mathbf{x}) & \cdots & P_N(\mathbf{x}) \end{bmatrix} \begin{bmatrix} u_{x,1}^2 & u_{x,1}u_{x,2} & \cdots & u_{x,1}u_{x,N} \\ u_{x,2}u_{x,1} & u_{x,2}^2 & \cdots & u_{x,2}u_{x,N} \\ \vdots & \ddots & \ddots & \vdots \\ u_{x,N}u_{x,1} & u_{x,N}u_{x,2} & \cdots & u_{x,N}^2 \end{bmatrix} \begin{bmatrix} P_1(\mathbf{x}) \\ P_2(\mathbf{x}) \\ \vdots \\ P_N(\mathbf{x}) \end{bmatrix} w(\mathbf{x}) d\mathbf{x} \\
& = \int_{\mathcal{T}^2} \begin{bmatrix} P_1(\mathbf{x}) & P_2(\mathbf{x}) & \cdots & P_N(\mathbf{x}) \end{bmatrix} \begin{bmatrix} u_{x,1} \sum_{j=1}^N u_{x,j} P_j(\mathbf{x}) \\ u_{x,2} \sum_{j=1}^N u_{x,j} P_j(\mathbf{x}) \\ \vdots \\ u_{x,N} \sum_{j=1}^N u_{x,j} P_j(\mathbf{x}) \end{bmatrix} w(\mathbf{x}) d\mathbf{x} \\
& = \int_{\mathcal{T}^2} w(\mathbf{x}) \sum_{k=1}^N P_k(\mathbf{x}) u_{x,k} \sum_{j=1}^N u_{x,j} P_j(\mathbf{x}) d\mathbf{x} = \int_{\mathcal{T}^2} w(\mathbf{x}) \sum_{k=1}^N \sum_{j=1}^N u_{x,k} u_{x,j} P_k(\mathbf{x}) P_j(\mathbf{x}) d\mathbf{x} \\
& = \int_{\mathcal{T}^2} \sum_{k=1}^N u_{x,k}^2 P_k^2(\mathbf{x}) w(\mathbf{x}) d\mathbf{x} = \sum_{k=1}^N u_{x,k}^2 \int_{\mathcal{T}^2} P_k^2(\mathbf{x}) dw(\mathbf{x}) \quad (\text{now we add back in } y \text{ part}) \\
& = \sum_{k=1}^N u_{x,k}^2 + \sum_{k=1}^N u_{y,k}^2 \\
& = \mathbf{u}^T (\tilde{\mathcal{D}}_x^T \tilde{\mathcal{D}}_x + \tilde{\mathcal{D}}_y^T \tilde{\mathcal{D}}_y) \mathbf{u} = \tilde{\mathbf{c}}_{\frac{1}{f}}^T \tilde{\mathbf{c}}_{\frac{1}{f}}, \quad \blacksquare
\end{aligned}$$

where we've used the mutual orthonormality relations, which persist even within a subspace of homogeneous polynomials of degree  $m$ , to eliminate all off-diagonal terms in the sum. Note, not all orthonormal bases are created equal, and we normally would have to retain many outer product terms in the sum because orthonormality between vectors of the same subspace is not a given.

Thus, we want to solve the following quadratic program with equality constraints:

$$(1.13) \quad \min_{\mathbf{u} \in \mathbb{R}^N} F(\mathbf{u}) = \mathbf{u}^T G \mathbf{u} - \tilde{\mathbf{c}}_1^T \tilde{\mathbf{c}}_{\frac{1}{f}} \\ \text{subject to} \quad \mathbf{u} \cdot \mathbf{P}(\mathbf{x}) = 0, \quad \mathbf{x} \in \partial\Omega,$$

where  $G = \tilde{\mathcal{D}}_x^T \tilde{\mathcal{D}}_x + \tilde{\mathcal{D}}_y^T \tilde{\mathcal{D}}_y \in \mathbb{R}^{N \times N}$  is symmetric and positive semi-definite<sup>1</sup>. The gradient and Hessian of  $F$  are given by

$$(1.14) \quad \nabla_{\mathbf{u}} F(\mathbf{u}) = 2G\mathbf{u},$$

$$(1.15) \quad \nabla_{\mathbf{u}}^2 F(\mathbf{u}) = 2G,$$

and we still need to consider the constraint equation. We take the approach of eliminating the constraints by solving a the minimization problem on auxiliary variables. That is, suppose  $\mathbf{P}(\mathbf{x}) \in \mathbb{R}^{M \times N}$ ,  $M > N$ , has rank  $p$ . We can compute the *full SVD* of  $\mathbf{P}(\mathbf{x}) = U\Sigma V^T$ . Then, the last  $n - p$  rows of  $V^T$  form a basis for the  $\mathcal{N}(\mathbf{P}(\mathbf{x}))$ .

That is,  $V^T = \begin{bmatrix} V_p^T \\ V_{n-p}^T \end{bmatrix}$ , and any solution  $\mathbf{u}$  satisfying the equality constraints can be expressed as

$$(1.16) \quad \mathbf{u} = V_{n-p} \mathbf{u}_{\text{eq}},$$

Hence, we can solve

$$(1.17) \quad \min_{\mathbf{u}_{\text{eq}} \in \mathbb{R}^N} F(\mathbf{u}_{\text{eq}}) = \mathbf{u}_{\text{eq}}^T V_{n-p}^T G V_{n-p} \mathbf{u}_{\text{eq}} - \tilde{\mathbf{c}}_1^T \tilde{\mathbf{c}}_{\frac{1}{f}}$$

and finally find  $\mathbf{u}$  by computing as in (1.16). Note, the gradient and Hessian of the auxiliary problem are given by

$$(1.18) \quad \nabla_{\mathbf{u}_{\text{eq}}} F(\mathbf{u}_{\text{eq}}) = 2V_{n-p}^T G V_{n-p} \mathbf{u}_{\text{eq}},$$

$$(1.19) \quad \nabla_{\mathbf{u}}^2 F(\mathbf{u}) = 2V_{n-p}^T G V_{n-p}.$$

Note, we can find  $p$  by the number of singular values above some small threshold that we choose (near  $\varepsilon_{\text{mach}}$ ) so that it represents numerical rank.

*Remark 1.1.* There is a ton of material in chapter 12 and 16 of the Nocedal/Wright optimization book. I believe I can prove properties on minimizers to (1.13) (though unnecessary) and maybe use a direct method therein. There still remains the problem of initialization..perhaps we just initialize from the null space, or solve a linearized problem?

*Remark 1.2.* By making use of a weighted basis  $\tilde{\mathbf{P}}(\mathbf{x})$ , and expressing our solution coefficients and differential operators in terms of this weighted basis, we can alternatively eliminate the constraint equation and solve a completely unconstrained quadratic optimization problem **I think I need to do some more symbolic computation to figure out the form of the RHS and/or additional conditions  $u$  must first satisfy so that we solve the same problem. I implemented that weighted basis, but I don't think I will get things to work properly, or rather, I will not continue trying.** We provide more details in the next section.

<sup>1</sup>Note,  $\forall \mathbf{x} \in \mathbb{R}_*^{n+}$ ,  $\mathbf{x}^T G \mathbf{x} = \mathbf{x}^T \tilde{\mathcal{D}}_x^T \tilde{\mathcal{D}}_x \mathbf{x} + \mathbf{x}^T \tilde{\mathcal{D}}_y^T \tilde{\mathcal{D}}_y \mathbf{x} = \|\tilde{\mathcal{D}}_x \mathbf{x}\|^2 + \|\tilde{\mathcal{D}}_y \mathbf{x}\|^2 \geq 0$ . We have equality only for any  $\mathbf{x} = \alpha \mathbf{e}_1$ , as derivatives of the constant polynomial are 0.

**1.2. A variational approach.** Let's try framing the problem from the perspective of infinite dimensional optimization. Thus far, my efforts have fallen under the discretize-then-optimize approach for solving non-linear PDEs. So, let's flip that and consider the optimize-then-discretize approach. The first order conditions for optimality of the functional may serve as a way to initialize the problem for the discretized non-linear optimization call. We want to solve

$$(1.20) \quad \begin{aligned} & \min_{u \in L_2(\mathcal{T}^2, w^{(a,b,c)})} \mathcal{L}(u), \\ & \text{s.t. } u = q \text{ on } \partial\mathcal{T}^2, \end{aligned}$$

where  $\mathcal{L} : L_2(\mathcal{T}^2, w^{(a,b,c)}) \rightarrow \mathbb{R}^+$  is defined by

$$(1.21) \quad \mathcal{L}(u) := \int_{\mathcal{T}^2} \left[ |\nabla_{\mathbf{x}} u|^2 - \left(\frac{1}{f}\right)^2 \right]^{\frac{1}{2}} d\mathbf{x},$$

and  $u, f$  are as in (1.1), restricted to  $L_2(\mathcal{T}^2, \mathcal{W}^{(a,b,c)})$ . The steps here are to first take the Gateaux derivative of  $\mathcal{L}$  as

$$\delta\mathcal{L}(u, \hat{u}) = \partial\varepsilon\mathcal{L}(u + \varepsilon\hat{u})|_{\varepsilon=0} = 0,$$

where  $\hat{u} \in \mathcal{H}_0^1(\mathcal{T}^2)$ . This gives the necessary condition a minimizer of  $\mathcal{L}$  must satisfy. Before proceeding, recall Green's identity for  $u, v \in H^1(\mathcal{T}^2)$ , i.e. a multidimensional integration by parts formula:

$$(1.22) \quad \int_{\mathcal{T}^2} \nabla_{\mathbf{x}} u \cdot v d(\mathbf{x}) = \int_{\partial\mathcal{T}^2} (uv) \cdot \mathbf{n} dS(\mathbf{x}) - \int_{\mathcal{T}^2} u \nabla_{\mathbf{x}} \cdot v d(\mathbf{x}).$$

Accordingly, we have

$$\begin{aligned} 0 &= \partial\varepsilon\mathcal{L}(u + \varepsilon\hat{u})|_{\varepsilon=0} \\ &= \partial_\varepsilon \left[ \int_{\mathcal{T}^2} \left( (u_x^2 + u_y^2) + 2\varepsilon(\hat{u}_x u_x + \hat{u}_y u_y) + \varepsilon^2(\hat{u}_x^2 + \hat{u}_y^2) - \left(\frac{1}{f}\right)^2 \right)^{\frac{1}{2}} d(\mathbf{x}) \right] \Big|_{\varepsilon=0} \\ &= \left[ \frac{1}{2} \int_{\mathcal{T}^2} \frac{2(\hat{u}_x u_x + \hat{u}_y u_y) + 2\varepsilon(\hat{u}_x^2 + \hat{u}_y^2)}{\left( (u_x^2 + u_y^2) + 2\varepsilon(\hat{u}_x u_x + \hat{u}_y u_y) + \varepsilon^2(\hat{u}_x^2 + \hat{u}_y^2) - \left(\frac{1}{f}\right)^2 \right)^{\frac{1}{2}}} d(\mathbf{x}) \right] \Big|_{\varepsilon=0}. \end{aligned}$$

The first variation is then given by

$$(1.23) \quad \delta\mathcal{L}(u, \hat{u}) = \int_{\mathcal{T}^2} \frac{\nabla_{\mathbf{x}} u \cdot \nabla_{\mathbf{x}} \hat{u}}{\sqrt{|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2}}} d(\mathbf{x}).$$

We can apply (1.22) with  $u \leftarrow \hat{u}$  and  $v \leftarrow \frac{\nabla_{\mathbf{x}} u}{\sqrt{|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2}}}$  to derive the strong (PDE) form of the first order necessary condition a minimizer of  $\mathcal{L}$  must satisfy. Using the fact that  $\hat{u}$  is 0 on the boundary, we have

$$\begin{aligned} \delta\mathcal{L}(u, \hat{u}) &= \int_{\partial\mathcal{T}^2} \hat{u} \frac{\nabla_{\mathbf{x}} u}{\sqrt{|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2}}} \cdot \mathbf{n} dS(\mathbf{x}) - \int_{\mathcal{T}^2} \hat{u} \nabla_{\mathbf{x}} \cdot \frac{\nabla_{\mathbf{x}} u}{\sqrt{|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2}}} d(\mathbf{x}) \\ (1.24) \quad &= - \int_{\mathcal{T}^2} \hat{u} \nabla_{\mathbf{x}} \cdot \frac{\nabla_{\mathbf{x}} u}{\sqrt{|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2}}} d(\mathbf{x}) = 0 \end{aligned}$$

Furthermore, the choice of  $\hat{u}$  is arbitrary. Thus, we find that a minimizer  $u$  satisfies the following PDE in  $\mathcal{T}^2$ ;

$$(1.25) \quad \begin{aligned} -\nabla_{\mathbf{x}} \cdot \left( \frac{\nabla_{\mathbf{x}} u}{\sqrt{|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2}}} \right) &= 0, \quad \mathbf{x} \in \mathcal{T}^2, \\ u &= q, \quad \mathbf{x} \in \partial\mathcal{T}^2. \end{aligned}$$

Next, we compute the Frechet derivative  $\delta^2 \mathcal{L}(u, \hat{u}, \tilde{u})$ , where  $\tilde{u} \in H_0^1(\mathcal{T}^2)$ . Letting  $u \leftarrow u + \varepsilon \tilde{u}$  in (1.23), we have

$$(1.26) \quad \begin{aligned} \delta^2 \mathcal{L}(u, \hat{u}, \tilde{u}) &= \partial_{\varepsilon} \delta \mathcal{L}(u + \varepsilon \tilde{u}, \hat{u}) \Big|_{\varepsilon=0} \\ &= \left( \partial_{\varepsilon} \int_{\mathcal{T}^2} \frac{\nabla_{\mathbf{x}} u \cdot \nabla_{\mathbf{x}} \hat{u} + \varepsilon \nabla_{\mathbf{x}} \tilde{u} \cdot \nabla_{\mathbf{x}} \hat{u}}{\sqrt{-\frac{1}{f^2} + \nabla_{\mathbf{x}} u \cdot \nabla_{\mathbf{x}} u + 2\varepsilon \nabla_{\mathbf{x}} u \cdot \nabla_{\mathbf{x}} \tilde{u} + \varepsilon^2 \nabla_{\mathbf{x}} \tilde{u} \cdot \nabla_{\mathbf{x}} \tilde{u}}} d(\mathbf{x}) \right) \Big|_{\varepsilon=0} \\ &= \int_{\mathcal{T}^2} \left( \frac{\nabla_{\mathbf{x}} \tilde{u} \cdot \nabla_{\mathbf{x}} \hat{u}}{\sqrt{-\frac{1}{f^2} + |\nabla_{\mathbf{x}} u|^2}} - \frac{(\nabla_{\mathbf{x}} u \cdot \nabla_{\mathbf{x}} \hat{u})(\nabla_{\mathbf{x}} u \cdot \nabla_{\mathbf{x}} \tilde{u})}{(-\frac{1}{f^2} + |\nabla_{\mathbf{x}} u|^2)^{\frac{3}{2}}} \right) d(\mathbf{x}) \end{aligned}$$

The weak form of the Newton system at a point  $u^k \in H_0^1(\mathcal{T}^2)$  is then independent of  $\hat{u}$ , and given by

$$(1.27) \quad \begin{aligned} &\text{Find } \tilde{u} \in H_0^1(\mathcal{T}^2) \text{ s.t.} \\ &\delta^2 \mathcal{L}(u^k)(\hat{u}, \tilde{u}) = -\delta \mathcal{L}(u^k)(\hat{u}), \\ &\text{and set } u^{k+1} = u^k + \tilde{u}, \\ &\text{with } u^0 = q \text{ on } \partial\mathcal{T}^2. \end{aligned}$$

To derive the strong form of the Newton system, we proceed as for the first variation using integration by parts formula. Let  $u \leftarrow \hat{u}$  and  $v \leftarrow \frac{\nabla_{\mathbf{x}} \tilde{u}}{\sqrt{|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2}}}$  for the first term in (1.26). For the second term therein, let  $u \leftarrow \hat{u}$  and  $v \leftarrow \frac{\nabla_{\mathbf{x}} \tilde{u} |\nabla_{\mathbf{x}} u|^2}{(|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2})^{\frac{3}{2}}}$ . Using the fact that  $\hat{u} \in H_0^1(\mathcal{T}^2)$ , and incorporating the first variation from (1.23) as needed in (1.27), we find the Newton system is equivalently expressed as

$$\int_{\mathcal{T}^2} \hat{u} \left[ \nabla_{\mathbf{x}} \cdot \left( \frac{\nabla_{\mathbf{x}} \tilde{u} |\nabla_{\mathbf{x}} u|^2}{(|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2})^{\frac{3}{2}}} - \frac{\nabla_{\mathbf{x}} \tilde{u} + \nabla_{\mathbf{x}} u}{\sqrt{|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2}}} \right) \right] d(\mathbf{x}) = 0$$

Then, by the arbitrariness of  $\hat{u}$ , we find the strong form of the Newton system is

$$(1.28) \quad \begin{aligned} &\text{Find } \tilde{u} \in H_0^1(\mathcal{T}^2) \text{ s.t.} \\ &\nabla_{\mathbf{x}} \cdot \left[ \nabla_{\mathbf{x}} \tilde{u} \left( \frac{|\nabla_{\mathbf{x}} u^k|^2}{(|\nabla_{\mathbf{x}} u^k|^2 - \frac{1}{f^2})^{\frac{3}{2}}} - \frac{1}{\sqrt{|\nabla_{\mathbf{x}} u^k|^2 - \frac{1}{f^2}}} \right) \right] = \nabla_{\mathbf{x}} \cdot \left( \frac{\nabla_{\mathbf{x}} u^k}{\sqrt{|\nabla_{\mathbf{x}} u^k|^2 - \frac{1}{f^2}}} \right) \\ &\text{and set } u^{k+1} = u^k + \tilde{u}, \\ &\text{with } u^0 = q \text{ on } \partial\mathcal{T}^2. \end{aligned}$$

By choosing  $u^0 = q$  on  $\partial\mathcal{T}^2$ , and setting  $u^{k+1} = u^k + \tilde{u}$ , we have an infinite-dimensional newton stepper for solving the Eikonal problem. To avoid the trivial solution  $u \equiv 0$  when  $q \equiv 0$ , we can try this with

$$u^0 = w^{(3/2, 3/2, 3/2)}(x, y) = xy(1 - x - y),$$

and use the weighted basis for differentiation in coefficient space. We iterate until the relative step difference is within some tolerance, in the  $L_2(\mathcal{T}^2, w)$  norm. I will give details on the weighted Laplace operator momentarily.

---

**Algorithm 1.1** Netwon updates for solving (1.1)

---

**Require:**  $\text{tol} < 1$

$u^k \leftarrow xy(1 - x - y)$

$\tilde{u} \leftarrow 1$

**while**  $(|\nabla_{\mathbf{x}} u^k|^2 - (\frac{1}{f})^2 > 0)$  and  $\|\tilde{u}\|_{L^2(\mathcal{T}^2, \tilde{w})} > \text{tol}$  **do**

$\tilde{u} \leftarrow$  solution to (1.28)

$u^{k+1} \leftarrow u^k + \tilde{u}$

$u^k \leftarrow u^{k+1}$

**end while**

---

Note, Algorithm 1.1 as well as the preceding derivations of weak and strong forms all assume that  $|\nabla_{\mathbf{x}} u^k|^2 - (\frac{1}{f})^2 > 0$  for any  $k$ . To prevent the need for this check, we can consider the modified (though equivalent) functional

$$(1.29) \quad \mathcal{L}(u) = \int_{\mathcal{T}^2} [(|\nabla_{\mathbf{x}} u|^2 - \frac{1}{f^2})^2]^{\frac{1}{4}}$$

Even though (1.29) is equivalent to (1.21), the latter does not necessarily ensure that the range of  $\mathcal{L}$  is  $\mathbb{R}^+$  without specifying the appropriate branch of  $\sqrt{\cdot}$  to consider. The modified functional does this without us having to deal with absolute values or the  $\text{sgn}$  function. Any arguments raised to the  $\frac{1}{2}$  or  $\frac{3}{2}$  powers, as they are in (1.28), can simply be squared and square root-ed before exponentiation. That is, the numerical implementation will include such a transformation to those arguments (this might work? we'll see). Expressing (1.28) in modal form is our next challenge.



## 2. The weighted Jacobi basis. The weighted basis

$$(2.1) \quad \tilde{\mathbf{P}}^{(a,b,c)}(x, y) = x^{a-\frac{1}{2}} y^{b-\frac{1}{2}} (1-x-y)^{c-\frac{1}{2}} \mathbf{P}^{(a,b,c)}$$

will prove useful for incorporating boundary conditions to the discrete operator, while preserving its sparsity. Note, we are simply multiplying the original basis by the weight function  $W^{(a,b,c)}(\mathbf{x})$  which forms the measure under which they are orthogonal. We need the following weighted ladder operators for differentiation in the weighted basis:

$$(2.2) \quad \begin{aligned} -(2k+b+c+1)h_{k,n}^{(a,b,c)} \partial_x \tilde{P}_{k,n}^{(a,b,c)} &= (k+c)(n-k+1)h_{k,n+1}^{(a-1,b,c-1)} \tilde{P}_{k,n+1}^{(a-1,b,c-1)} \\ &\quad + (k+1)(n-k+a)h_{k+1,n+1}^{(a-1,b,c-1)} \tilde{P}_{k+1,n+1}^{(a-1,b,c-1)} \end{aligned}$$

$$(2.3) \quad h_{k,n}^{(a,b,c)} \partial_y \tilde{P}_{k,n}^{(a,b,c)} = -(k+1)h_{k+1,n+1}^{(a,b-1,c-1)} \tilde{P}_{k+1,n+1}^{(a,b-1,c-1)}.$$

If  $\mathbf{f}$  are the coefficients of  $f$  under the  $(a, b, c)$  basis, then we have sparse banded matrices  $\mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)}$ ,  $\mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)}$  such that

$$(2.4) \quad \begin{aligned} \partial_x [x^{a-\frac{1}{2}} y^{b-\frac{1}{2}} (1-x-y)^{c-\frac{1}{2}} f(x, y)] &= \partial_x (\tilde{\mathbf{P}}^{(a,b,c)}(x, y)^T \mathbf{f}) \\ &= \tilde{\mathbf{P}}^{(a-1,b,c-1)}(x, y)^T \mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)} \mathbf{f}, \end{aligned}$$

$$(2.5) \quad \begin{aligned} \partial_y [x^{a-\frac{1}{2}} y^{b-\frac{1}{2}} (1-x-y)^{c-\frac{1}{2}} f(x, y)] &= \partial_y (\tilde{\mathbf{P}}^{(a,b,c)}(x, y)^T \mathbf{f}) \\ &= \tilde{\mathbf{P}}^{(a,b-1,c-1)}(x, y)^T \mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)} \mathbf{f} \end{aligned}$$

**2.0.1. Lowering operators.** Additionally, we have lowering operators which relate  $xP_{k,n}^{(a,b,c)}$  to  $P_{k,n}^{(a-1,b,c)}$  and  $yP_{k,n}^{(a,b,c)}$  to  $P_{k,n}^{(a,b-1,c)}$ . These are given by the following recurrences:

$$(2.6) \quad \begin{aligned} (2n+a+b+c+2)h_{k,n}^{(a,b,c)} xP_{k,n}^{(a,b,c)} &= (n-k+a)h_{k,n}^{(a-1,b,c)} P_{k,n}^{(a-1,b,c)} \\ &\quad + (n-k+1)h_{k,n+1}^{(a-1,b,c)} P_{k,n+1}^{(a-1,b,c)}, \end{aligned}$$

$$(2.7) \quad \begin{aligned} (2k+b+c+1)(2n+a+b+c+2)h_{k,n}^{(a,b,c)} yP_{k,n}^{(a,b,c)} &= (k+b)(n+k+b+c+1)h_{k,n}^{(a,b-1,c)} P_{k,n}^{(a,b-1,c)} \\ &\quad - (k+1)(n-k+a)h_{k+1,n}^{(a,b-1,c)} P_{k+1,n}^{(a,b-1,c)} \\ &\quad - (k+b)(n-k+1)h_{k,n+1}^{(a,b-1,c)} P_{k,n+1}^{(a,b-1,c)} \\ &\quad + (k+1)(n+k+a+b+c+2)h_{k+1,n+1}^{(a,b-1,c)} P_{k+1,n+1}^{(a,b-1,c)}. \end{aligned}$$

So, there are sparse banded matrices  $\mathcal{L}_{(a,b,c)}^{(a-1,b,c)}$  and  $\mathcal{L}_{(a,b,c)}^{(a,b-1,c)}$  corresponding to multiplication by  $x$  and  $y$ , respectively, such that

$$(2.9) \quad xf(x, y) = x\mathbf{P}^{(a,b,c)}(x, y)^T \mathbf{f} = \mathbf{P}^{(a-1,b,c)}(x, y)^T \mathcal{L}_{(a,b,c)}^{(a-1,b,c)} \mathbf{f},$$

$$(2.10) \quad yf(x, y) = y\mathbf{P}^{(a,b,c)}(x, y)^T \mathbf{f} = \mathbf{P}^{(a,b-1,c)}(x, y)^T \mathcal{L}_{(a,b,c)}^{(a,b-1,c)} \mathbf{f}.$$

**2.0.2. Discrete differential operators.** We will construct the modal Eikonal operator which operates on coefficients under the basis  $\tilde{\mathbf{P}}^{(3/2,3/2,3/2)}$  and produces coefficients under  $\mathbf{P}^{(3/2,3/2,3/2)}$ . Note, the conversion from the weighted to regular

Jacobi basis depends on our choice of parameters  $a = b = c = 3/2$ . With  $\mathbf{f}$  the coefficients of  $f(x, y)$  in the  $(a, b, c)$  basis,

$$\begin{aligned}\partial_x[W^{(a,b,c)}(x, y)f(x, y)] &= \tilde{\mathbf{P}}^{(a-1,b,c-1)}(x, y)^T \mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)} \mathbf{f} \\ &= x^{a-\frac{3}{2}} y^{b-\frac{3}{2}} (1-x-y)^{c-\frac{3}{2}} (y \mathbf{P}^{(a-1,b,c-1)}(x, y))^T (\mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)} \mathbf{f}) \\ &= x^{a-\frac{3}{2}} y^{b-\frac{3}{2}} (1-x-y)^{c-\frac{3}{2}} \mathbf{P}^{(a-1,b-1,c-1)}(x, y)^T (\mathcal{L}_{(a-1,b,c-1)}^{(a-1,b-1,c-1)} \mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)} \mathbf{f}), \\ \partial_y[W^{(a,b,c)}(x, y)f(x, y)] &= \tilde{\mathbf{P}}^{(a,b-1,c-1)}(x, y)^T \mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)} \mathbf{f} \\ &= x^{a-\frac{3}{2}} y^{b-\frac{3}{2}} (1-x-y)^{c-\frac{3}{2}} (x \mathbf{P}^{(a,b-1,c-1)}(x, y))^T (\mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)} \mathbf{f}) \\ &= x^{a-\frac{3}{2}} y^{b-\frac{3}{2}} (1-x-y)^{c-\frac{3}{2}} \mathbf{P}^{(a-1,b-1,c-1)}(x, y)^T (\mathcal{L}_{(a,b-1,c-1)}^{(a-1,b-1,c-1)} \mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)} \mathbf{f})\end{aligned}$$

When  $a = b = c = 3/2$ , we have

$$\begin{aligned}\partial_x[W^{(a,b,c)}(x, y)f(x, y)] &= \mathbf{P}^{(a-1,b-1,c-1)}(x, y)^T (\mathcal{L}_{(a-1,b,c-1)}^{(a-1,b-1,c-1)} \mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)} \mathbf{f}), \\ \partial_y[W^{(a,b,c)}(x, y)f(x, y)] &= \mathbf{P}^{(a-1,b-1,c-1)}(x, y)^T (\mathcal{L}_{(a,b-1,c-1)}^{(a-1,b-1,c-1)} \mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)} \mathbf{f}),\end{aligned}$$

We need to see what auxiliary problems we must solve, as the nonlinearity of (1.1) leads to cross-terms that must cancel if we hope that a solution to the weighted problem also solves the unweighted problem. In the bulk, we have

$$[\partial_x(Wu)]^2 + [\partial_y(Wu)]^2 = W^2[u_x^2 + u_y^2] + u^2(W_x + W_y) + 2uW(W_x u_x + W_y u_y),$$

and we need the second two terms in the equation to cancel. [There's something here we can do by integrating, and finding the additional conditions  \$u\$  must satisfy. I think it would become a solution for the initial guess of  \$u\$ . Essentially, we know  \$W^2\[u\_x^2 + u\_y^2\] = W^2/f^2\$ , since we assume that  \$u\$  solve \(1.1\). Then, we can enforce that](#)

$$\int_{\mathcal{T}^2} u^2(W_x + W_y) + 2uW(W_x u_x + W_y u_y) = 0.$$

[There may be some simplification that occurs if we plug in the Jacobi expansion of  \$u\$  .. idk](#)

In the weighted basis, second partials acting on function modes can be constructed with the promotion and unweighted ladder operator as done in Section 2.0.2.

(2.11)

$$\partial_{xx}f(x, y) = \mathbf{P}^{(a,b,c)}(x, y)^T \left( \mathcal{K}_{(a,b-1,c)}^{(a,b,c)} \mathcal{D}_{x,(a-1,b-1,c-1)}^{(a,b-1,c)} \mathcal{L}_{(a-1,b,c-1)}^{(a-1,b-1,c-1)} \mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)} \mathbf{f} \right),$$

(2.12)

$$\partial_{yy}f(x, y) = \mathbf{P}^{(a,b,c)}(x, y)^T \left( \mathcal{K}_{(a-1,b,c)}^{(a,b,c)} \mathcal{D}_{y,(a-1,b-1,c-1)}^{(a-1,b,c)} \mathcal{L}_{(a,b-1,c-1)}^{(a-1,b-1,c-1)} \mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)} \mathbf{f} \right),$$

(2.13)

$$\partial_{xy}f(x, y) = \mathbf{P}^{(a,b,c)}(x, y)^T \left( \mathcal{K}_{(a,b-1,c)}^{(a,b,c)} \mathcal{D}_{x,(a-1,b-1,c-1)}^{(a,b-1,c)} \mathcal{L}_{(a,b-1,c-1)}^{(a-1,b-1,c-1)} \mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)} \mathbf{f} \right)$$

$\Delta_W$ , the weighted Laplace operator, is then given by

$$\begin{aligned}(2.14) \quad \Delta_W &= \mathcal{K}_{(a,b-1,c)}^{(a,b,c)} \mathcal{D}_{x,(a-1,b-1,c-1)}^{(a,b-1,c)} \mathcal{L}_{(a-1,b,c-1)}^{(a-1,b-1,c-1)} \mathcal{W}_{x,(a,b,c)}^{(a-1,b,c-1)} \\ &\quad + \mathcal{K}_{(a-1,b,c)}^{(a,b,c)} \mathcal{D}_{y,(a-1,b-1,c-1)}^{(a-1,b,c)} \mathcal{L}_{(a,b-1,c-1)}^{(a-1,b-1,c-1)} \mathcal{W}_{y,(a,b,c)}^{(a,b-1,c-1)}.\end{aligned}$$

The cross partial terms are needed only when we include affine transformations from general triangles to the standard reference triangle, as they appear when chaining out derivatives of the parametrization.

**3. Quadrature generation for Jacobi polynomials on the Simplex.** Here, we detail the procedure for generating quadrature rules on the simplex. That is, given  $f : \mathcal{T}^d \rightarrow \mathbb{R}$ , we seek a set of  $N$   $(d+1)$ -tuples  $(\mathbf{X}_1, \dots, \mathbf{X}_d, \mathbf{W})$  such that

$$(3.1) \quad \begin{aligned} & \int_{\mathcal{T}^d} f(x_1, \dots, x_d) w^{(\mathbf{a})}(x_1, \dots, x_d) dx_1 \cdots dx_d \\ & \approx \sum_{j=1}^N f(x_{j1}, \dots, x_{jd}) w_j, \end{aligned}$$

where  $w^{(\mathbf{a})}(x_1, \dots, x_d)$  is the normalized Jacobi weight function on  $\mathcal{T}^d$ , and the approximation is exact for  $M > N$  polynomials

Suppose that  $\mathbf{P}(\mathbf{x}) \in \mathbb{R}^N$ , with

$$N = \dim(\Pi_{n-1}^d) = \binom{n-1+d}{n-1},$$

as in (1.5) (in which we had  $d = 2$ ), and  $\mathbf{x} \in \mathbb{R}^d$ . The orthonormal 3-term relation for Jacobi polynomials on the simplex is given by

$$(3.2) \quad x_i \mathbb{P}_n = A_{n,i} \mathbb{P}_{n+1} + B_{n,i} \mathbb{P}_n + A_{n-1,i}^T \mathbb{P}_{n-1}, \quad i = 1, \dots, d$$

$$(3.3) \quad A_{n,i} = \langle x_i \mathbb{P}_n, \mathbb{P}_{n+1}^T \rangle \in \mathbb{R}^{r_n^d \times r_{n+1}^d},$$

$$(3.4) \quad B_{n,i} = \langle x_i \mathbb{P}_n, \mathbb{P}_n^T \rangle \in \mathbb{R}^{r_n^d \times r_n^d},$$

where  $\langle \cdot \rangle$  is the weighted  $L^2$  inner product. The matrix recurrence generates an analog of the Jacobi matrix from 1D, except now there are  $d$  of them - one for each dimension:

$$(3.5) \quad \mathcal{J}_{n,i} = \begin{bmatrix} B_{0,i} & A_{0,i} & & & \\ A_{0,i}^T & B_{1,i} & A_{1,i} & & \\ & A_{1,i}^T & \ddots & \ddots & \\ & & \ddots & B_{n-2,i} & A_{n-2,i} \\ & & & A_{n-2,i}^T & B_{n-1,i} \end{bmatrix}, \quad i = 1, \dots, d$$

with  $\mathcal{J}_{n,i} \in \mathbb{R}^{N \times N}$  symmetric. By definition, we see that the Jacobi matrices satisfy

$$(3.6) \quad x_i \mathbf{P} = \mathcal{J}_{n,i} \mathbf{P} + \begin{bmatrix} \mathbf{0}_{(N-r_{n-1}^d) \times 1} \\ A_{n-1,i} \mathbb{P}_n \end{bmatrix}, \quad i = 1, \dots, d.$$

We hope for a way to generalize the univariate Golub-Welsch algorithm for determining Gaussian quadrature from orthogonal polynomials on the line to the multivariate case on the simplex. We say that  $\Lambda = (\lambda_1, \dots, \lambda_d)$  is a *joint eigenvalue* of  $J_{n,1}, \dots, J_{n,d}$  if there is a vector  $\xi \neq 0$  such that  $J_{n,i} \xi = \lambda_i \xi$  for  $i = 1, \dots, d$ . The vector  $\xi$  is the *joint eigenvector*. If there are  $N$  distinct joint eigenvalues, then the weights and abscissa for the Gaussian quadrature are as in 1D - the nodes are the joint eigenvalues, and the weights are the square of the first component of the corresponding joint eigenvector. However, Theorem 3.7.5 of Dunkl-Xu states that a necessary and sufficient condition for this to occur is the following commutativity relation

$$(3.7) \quad A_{n-1,i} A_{n-1,j}^T = A_{n-1,j} A_{n-1,i}^T, \quad 1 \leq i, j \leq d$$

This is equivalent to stating that  $J_{n,i}$  are a commuting family, and hence, simultaneously diagonalizable. By inspection, we see that the coefficient matrices in the recurrence for Jacobi polynomials on the triangle **do not**, in general, satisfy the commutativity relation. Thus, *there do not exist minimal Gaussian quadrature rules on the Triangle using the classic orthogonal polynomials.*

We need an accurate quadrature on the triangle, as the convergence of any modal solver depends highly on accurate representations of the right-hand-side in the modal basis. While  $J_{n,i}$  may not commute, there is a connection between their spectrums and non-minimal Gaussian-like quadrature rules on  $\mathcal{T}^d$ . Vioreanu and Rokhlin show that the joint eigenvalues of the combined complex operator

$$(3.8) \quad \mathcal{J}_n = \mathcal{J}_{n,1} + \iota \mathcal{J}_{n,2}$$

lie within  $\mathcal{T}^2$  (Theorem 3.4 in VRQUAD). The algorithm described in their work uses the constant unit weight function and gives an alternate way to evaluate  $\mathcal{J}_n$ . For the case of the weighted space  $L^2(\mathcal{T}, w_{a,b,c}W(x,y))$  with normalized weight (see (1.2)), the entries are

$$(3.9) \quad (\mathcal{J}_n)_{i,j} = \int_{\mathcal{T}} (x_1 + \iota x_2) P_j(x,y) P_i(x,y) w_{a,b,c} W(x,y) dx dy, \quad 0 \leq i, j \leq N-1,$$

where  $P_j$  is the  $j^{\text{th}}$  component of  $\mathbf{P} = (\mathbb{P}_0^T, \mathbb{P}_1^T, \dots, \mathbb{P}_{n-1}^T)^T = (P_0^0, P_0^1, P_1^1, \dots, P_n^n)^T \in \mathbb{R}^N$ . These are precisely the same operators defined in (3.8). However, this “complexification” trick is not available to us in dimensions  $d > 2$ , but there do exist algorithms for finding *approximate* joint diagonalizations of non-commuting matrix families.

**3.1. Approximate joint diagonalization of non-commuting symmetric matrices.** The approximate joint diagonalization problem for a set of real-valued symmetric matrices  $\{C^1, \dots, C^d\}$  is to find the orthogonal matrix  $U$  such that the matrices

$$(3.10) \quad F^k = UC^kU^T, \quad 1 \leq k \leq d$$

are approximately diagonal. Alternatively, we can find the diagonal matrices  $F^k$  such that  $C^k$  are approximately equal to  $U^T F^k U$ . The definition of “as diagonal as possible” leads to various formulations of the diagonalization problem in terms of minimizing an objective function. For example, solving

$$(3.11) \quad \min_{\substack{U \in \mathbb{R}^N \\ U^T U = U U^T = I}} \sum_{i \neq j} (F_{ij}^k)^2$$

is equivalent to minimizing the off-diagonal entries of  $F^k$  by orthogonal similarity transformations of  $C^k$ . On the other hand, solving

$$(3.12) \quad \min_{\substack{U, F^1, \dots, F^d \\ U^T U = U U^T = I}} \sum_{k=1}^d \|C^k - U^T F^k U\|_F^2$$

is finding the best representation of  $C^k$  in the basis of  $U$  - a subspace fitting approach. For both problems, we require orthogonality of  $U$ , as we want it to represent the “average eigenstructure” of  $C^k$ . For the Jacobi matrices  $J_{n,i}$ , this requirement is natural. We want a decomposition close to the optimal quadrature case of exact joint diagonalization. For this purpose, we use the algorithm specified in “Jacobi Angles for Simultaneous Diagonalization”. The joint-eigenvalues provide us with an initial guess to a subsequent optimization problem.

**3.2. Quadrature via roots.** We want to integrate the  $M$  polynomials up to total degree  $m-1$  exactly with  $m \geq n$  using only  $N$  nodes and weights. That is, with  $M = \dim(\Pi_{m-1}^d) = \binom{m-1+d}{m-1}$ , the integrals for which we want an exact quadrature are

$$(3.13) \quad \begin{aligned} \mathcal{I}_j &= \int_{\mathcal{T}^d} P_j(\mathbf{x}) w_{\mathbf{a}} W(\mathbf{x}) d(\mathbf{x}), \quad 0 \leq j \leq M-1 \\ &= \langle P_j, P_0 \rangle = \delta_{j0}, \end{aligned}$$

where we use mutual orthonormality under  $w_{a,b,c} W(x, y)$  (1.4) to evaluate them and arrive at the  $\delta_{j0}$  term. If  $\mathbf{X}_1, \dots, \mathbf{X}_d, \mathbf{W} \in \mathbb{R}^N$  are the nodes and weights,  $\mathcal{I} \in \mathbb{R}^M$  the above integrals, and  $\mathbf{P}(\mathbf{x}) \in \mathbb{R}^M$  (with  $\mathbf{x} \in \mathbb{R}^d$ ) our  $M > N$  polynomials, the objective function we aim to find a zero of is  $\mathbf{F} : \mathbb{R}^{(d+1)N} \rightarrow \mathbb{R}^M$  given by

$$(3.14) \quad \mathbf{F}(\mathbf{X}_1, \dots, \mathbf{X}_d, \mathbf{W}) = \mathbf{P}(\tilde{\mathbf{z}}) \mathbf{W} - \mathbf{e}_1, \quad \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \in \mathbb{R}^M, \quad \mathbf{P}(\tilde{\mathbf{z}}) \in \mathbb{R}^{M \times N},$$

where

$$(3.15) \quad \tilde{\mathbf{z}} = (\mathbf{X}_1, \dots, \mathbf{X}_d) \in \mathbb{R}^{dN}$$

Now, letting

$$(3.16) \quad \mathbf{z} = (\tilde{\mathbf{z}}, \mathbf{W}) \in \mathbb{R}^{(d+1)N},$$

we can use the Gauss-Newton iteration (where  $A^+$  is the Moore-Penrose pseudoinverse)

$$(3.17) \quad \mathbf{z}^{k+1} = \mathbf{z}^k - \alpha_k [\nabla(\mathbf{F}(\mathbf{z}^k))]^+ \mathbf{F}(\mathbf{z}^k),$$

to find roots of (3.14), where  $\alpha \in (0, 1)$ . As we show in Section 3.2.3, we have access to analytical formula for computing the derivatives of (3.14), which enables us to alternatively use the damped Newton iteration

$$(3.18) \quad \mathbf{z}^{k+1} = \mathbf{z}^k - \alpha [\mathcal{D}^2(\mathbf{F}(\mathbf{z}^k))]^{-1} \nabla(\mathbf{F}(\mathbf{z}^k)).$$

This iteration does not work well, as the Hessian for each component of the objective is not well conditioned, i.e. near singular. Using the convex problem is a better approach. However, the Gauss-Newton iteration gives rapid convergence towards the solution, albeit with violation of the constraints. It might be wise to use Gauss-Newton to further initialize the guess for the solution, before passing it to iteration (3.20). We also consider the Newton iteration for the equivalent problem of finding roots of the function  $f : \mathbb{R}^{(d+1)N} \rightarrow \mathbb{R}$  defined by

$$(3.19) \quad f(\mathbf{z}) := \|\mathbf{F}(\mathbf{z})\|_2^2,$$

with iteration

$$(3.20) \quad \mathbf{z}^{k+1} = \mathbf{z}^k - \alpha [\mathcal{D}^2(f(\mathbf{z}^k))]^{-1} \nabla(f(\mathbf{z}^k))$$

The choice of initial point  $\mathbf{z}^0 = (\tilde{\mathbf{z}}^0, \mathbf{W}_0)$  and control of the step length  $\alpha_k$  are vital for its convergence. Moreover, we'd like to incorporate inequality constraints on  $\mathbf{z}^k$

to enforce that quadrature nodes are interior to  $\mathcal{T}^2$ . Positivity of weights is also desirable, though not absolutely necessary for stability when the absolute value of each weight is not large. By casting the problem of finding roots of (3.14), (3.19) into a minimization problem, we can use techniques from constrained optimization to find acceptable quadratures.

First, we make a few observations about the objective function (3.19). Expanding out the norm in terms of the partitioning of the variable  $\mathbf{z}$  defined in (3.16), we have

$$(3.21) \quad f(\mathbf{z}) = \mathbf{W}^T \mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}}) \mathbf{W} - 2(\mathbf{1}_{N \times 1}^T \mathbf{W}) + 1.$$

Since  $M > N$ , and each of the  $M$  polynomials is independent,  $\mathbf{P}(\tilde{\mathbf{z}})$  has full column rank for any  $\tilde{\mathbf{z}} \in \mathbb{R}^{dN}$  with distinct points. This implies that  $\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})$  is positive definite, and (3.21) is a quadratic, strongly convex function in  $\mathbf{W}$ . The optimal  $\mathbf{W}$  is easily determined given the necessary and sufficient optimality condition for strongly convex functions; namely,  $\nabla_{\mathbf{W}} f(\mathbf{z}) = 0$ . Hence,

$$(3.22) \quad \begin{aligned} \nabla_{\mathbf{W}} f(\mathbf{z}) &= \mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}}) \mathbf{W} - \mathbf{1}_{N \times 1} = 0 \\ \Rightarrow \mathbf{W} &= [\mathbf{P}(\tilde{\mathbf{z}})^T (\mathbf{P}(\tilde{\mathbf{z}}))]^{-1} \mathbf{1}_{N \times 1}. \end{aligned}$$

The condition number of  $\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})$  grows as the polynomial order increases. Hence, we will need some kind of preconditioning before computing the weights. In particular, the pre-conditioning can be a derivative-free optimization on the condition number of  $\mathbf{P}^T(\tilde{\mathbf{z}})$  for a fixed  $\mathbf{W}$ , ensuring the value of  $f(\mathbf{z})$  does not exceed its initial value. I've already implemented this with NLOPT, and it works well. .

**3.2.1. Roots via minimization.** The optimization problem we would like to solve, stated in canonical form, is

$$(3.23) \quad \begin{aligned} \min_{\mathbf{z} \in \mathbb{R}^{(d+1)N}} f(\mathbf{z}) \quad \text{such that} \\ f_i(\mathbf{z}) \leq 0, \quad i = 1, \dots, (d+1)N \end{aligned}$$

where  $\mathbf{f} : \mathbb{R}^{(d+1)N} \rightarrow \mathbb{R}^{(d+1)N} := \begin{bmatrix} f_1(\mathbf{z}) \\ \vdots \\ f_{(d+1)N}(\mathbf{z}) \end{bmatrix}$  is given by

$$(3.24) \quad \mathbf{f}(\mathbf{z}) = G\mathbf{z} - \begin{bmatrix} \mathbf{0}_{dN \times 1} \\ \mathbf{1}_{N \times 1} \end{bmatrix}, \quad G = \begin{bmatrix} -I_1 & \times & \cdots & \times & \times \\ \times & \ddots & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ \times & \cdots & \cdots & -I_d & \times \\ I_1 & \cdots & \cdots & I_d & \times \end{bmatrix} \in \mathbb{R}^{(d+1)N \times (d+1)N},$$

and  $f$  is given by (3.19). The zero entries in the constraint matrix  $G$  are denoted by  $\times$ , and each  $I_1, \dots, I_d$  is an  $N \times N$  identity matrix. Specifically, each block row of the constraint equation  $\mathbf{f}(\mathbf{z})$  corresponds, respectively, to the following inequality constraints for  $j = 1 \dots N$ :

- (i)  $X_{1j} \geq 0, \dots, X_{dj} \geq 0$ , which restricts the solution to the non-negative  $dN$ -orthant .

- (ii)  $|\mathbf{X}_j|_1 \leq 1$ , which is the  $L_1$ -norm definition of the simplex  $\mathcal{T}^d$ , and means that no abscissa points should be outside of it.

Now, we can optimize over specific partitions of the variable  $\mathbf{z}$  independently, and formulate an equivalent problem. With  $\tilde{\mathbf{z}}$  as in (3.15), let

$$\begin{aligned}
 \tilde{f}(\tilde{\mathbf{z}}) &= \inf\{f(\tilde{\mathbf{z}}, \mathbf{W}) \mid \mathbf{W} \in \mathbb{R}^N\} \\
 &= f(\tilde{\mathbf{z}}, [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1} \mathbf{1}_{N \times 1}) \\
 &= \mathbf{1}_{1 \times N} [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1} \mathbf{1}_{N \times 1} - 2(\mathbf{1}_{1 \times N} [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1} \mathbf{1}_{N \times 1}) + 1 \\
 (3.25) \quad &= -\mathbf{1}_{1 \times N} [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1} \mathbf{1}_{N \times 1} + 1
 \end{aligned}$$

where we inserted the optimal  $\mathbf{W}$  found in (3.22). The problem (3.23) is then equivalent to

$$\begin{aligned}
 (3.26) \quad &\min_{\tilde{\mathbf{z}} \in \mathbb{R}^{dN}} \tilde{f}(\tilde{\mathbf{z}}) \quad \text{such that} \\
 &\tilde{f}_i(\tilde{\mathbf{z}}) \leq 0, \quad i = 1, \dots, (d+1)N
 \end{aligned}$$

where  $\tilde{\mathbf{f}} : \mathbb{R}^{dN} \rightarrow \mathbb{R}^{(d+1)N} := \begin{bmatrix} \tilde{f}_1(\tilde{\mathbf{z}}) \\ \vdots \\ \tilde{f}_{(d+1)N}(\tilde{\mathbf{z}}) \end{bmatrix}$  is given by

$$(3.27) \quad \tilde{\mathbf{f}}(\tilde{\mathbf{z}}) = G\tilde{\mathbf{z}} - \begin{bmatrix} \mathbf{0}_{dN \times 1} \\ \mathbf{1}_{N \times 1} \end{bmatrix}, \quad G = \begin{bmatrix} -I_1 & \times & \cdots & \times \\ \times & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ \times & \cdots & \cdots & -I_d \\ I_1 & \cdots & \cdots & I_d \end{bmatrix} \in \mathbb{R}^{(d+1)N \times dN},$$

For some reason, `logdet` and `trace` identities are coming to mind as useful for solving this problem - totally vague as of now. At the very least, we can compute derivatives of  $\tilde{f}(\tilde{\mathbf{z}})$ . Let  $A(\tilde{\mathbf{z}}) = \mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})$ . Then,  $\mathcal{D}(A^{-1})|_{\tilde{\mathbf{z}}} = -A(\tilde{\mathbf{z}})^{-1} \mathcal{D}(A(\tilde{\mathbf{z}})) A(\tilde{\mathbf{z}})^{-1}$ . This can be seen by taking the derivative of  $I = A(\tilde{\mathbf{z}})A(\tilde{\mathbf{z}})^{-1}$ , which is 0, and solving for the desired term.

Let  $L : \mathbb{R}^{dN} \times \mathbb{R}^{(d+1)N} \rightarrow \mathbb{R}$  denote the *Lagrangian* of the *primal objective* (3.26). With *dual variables*  $\boldsymbol{\lambda} \in \mathbb{R}^{(d+1)N}$ , it is given by

$$(3.28) \quad L(\tilde{\mathbf{z}}, \boldsymbol{\lambda}) = \tilde{f}(\tilde{\mathbf{z}}) + \boldsymbol{\lambda}^T \tilde{\mathbf{f}}(\tilde{\mathbf{z}}).$$

The *Lagrange dual function* of (??) is defined as the minimum value of the Lagrangian over  $\mathbf{z}$ :

$$(3.29) \quad g(\boldsymbol{\lambda}) = \inf_{\tilde{\mathbf{z}} \in \mathbb{R}^{dN}} L(\tilde{\mathbf{z}}, \boldsymbol{\lambda}).$$

The dual function gives a lower bound on the optimal value  $p^*$  of the optimization problem (3.26). It stands to reason that there may exist a “best” lower bound, obtainable through solving the *Lagrange dual problem*:

$$\begin{aligned}
 (3.30) \quad &\max_{\boldsymbol{\lambda} \in \mathbb{R}^{(d+1)N}} g(\boldsymbol{\lambda}) \\
 &\text{subject to } \boldsymbol{\lambda} \succeq 0
 \end{aligned}$$



The dual problem is a concave optimization problem, since the objective  $g(\boldsymbol{\lambda})$  is the pointwise infimum of a family of affine functions of  $\boldsymbol{\lambda}$  (they are the sums of hyperplanes with normal  $\boldsymbol{\lambda}$ ). This holds regardless of the convexity of the original objective  $\tilde{f}(\tilde{\mathbf{z}})$ . Thus, there exists a notion of the *duality gap*. If  $p^*$  is the optimal value of the primal problem (3.26), and  $d^*$  is that of the dual problem (3.30), then the lower bound property of the dual problem implies, regardless of the convexity of the primal objective, that

$$(3.31) \quad d^* \leq p^*.$$

The difference  $p^* - d^*$  is referred to as the duality gap. If the duality gap is zero, then we say that *strong duality* holds. That is, the bound obtained from the Lagrange dual function is tight, and solving the dual problem gives us a solution for the primal problem. The dual problem can be solved using the regular techniques of constrained convex optimization, though we have described it only for theoretical clarification.

If  $\tilde{\mathbf{z}}^*$  and  $(\boldsymbol{\lambda}^*)$  are primal and dual optimal points, respectively, with zero duality gap, then  $\tilde{\mathbf{z}}^*$  minimizes  $L(\tilde{\mathbf{z}}, \boldsymbol{\lambda}^*)$  over  $\tilde{\mathbf{z}}$ . Hence, the first order necessary optimality conditions must hold; namely, that the gradient of  $L$  must vanish:

$$(3.32) \quad \nabla \tilde{f}(\tilde{\mathbf{z}}^*) + \boldsymbol{\lambda}^{*T} \nabla \tilde{\mathbf{f}}(\tilde{\mathbf{z}}^*) = 0.$$

This leads to the following conditions characterizing the solution to any optimization problem for which strong duality holds, known as the Karush-Kuhn-Tucker (KKT) conditions, which we specify for (3.23):

$$(3.33) \quad \begin{aligned} \tilde{f}(\tilde{\mathbf{z}}^*) &\preceq 0, \\ \boldsymbol{\lambda}^* &\succeq 0, \\ \boldsymbol{\lambda}^{*T} \tilde{\mathbf{f}}(\tilde{\mathbf{z}}^*) &= 0, \\ \nabla \tilde{f}(\tilde{\mathbf{z}}^*) + \boldsymbol{\lambda}^{*T} \nabla \tilde{\mathbf{f}}(\tilde{\mathbf{z}}^*) &= 0. \end{aligned}$$

*Remark 3.1.* If an optimal primal dual pair is found,  $\mathbf{z}^*$  is not necessarily a zero of  $\mathbf{F}(\mathbf{z})$ . We have only satisfied the necessary KKT conditions. However, we may be closer to an actual root and will have maintained interior nodes and positive weights. We can use this new estimate  $\mathbf{z}^*$  as an initial point in a Gauss-Newton iteration that may relax  $\mathbf{F}(\mathbf{z})$  to zero quadratically.

We will solve (??) (when it is solvable) using a primal-dual interior-point method. Specifically, we employ a primal-dual barrier method. The initial iterate  $\mathbf{z}^0$  satisfies the inequality constraints in the abscissa partition, but the weights may not initially satisfy positivity, or sum to unity. For the primal-dual method, we need all inequality constraints to be inactive initially. Hence, we can either use an infeasible start newton method, or just pass the absolute value of the initial weights. Either way, we aim only to solve the KKT equations, and the optimal solution is not necessarily a root of (3.14).

**3.2.2. Primal-dual barrier method.** In the barrier method, we start by rewriting (??) in a form which incorporates the inequality constraints directly into the objective, as in

$$(3.34) \quad \begin{aligned} \min_{\mathbf{z} \in \mathbb{R}^{(d+1)N}} & f(\mathbf{z}) + \sum_{i=1}^{(d+2)N} I_-(f_i(\mathbf{z})) \\ \text{subject to} & \quad h_1(\mathbf{z}) = A\mathbf{z} - 1 = 0, \end{aligned}$$

where  $I_- : \mathbb{R} \rightarrow \mathbb{R}$  is the indicator function for the non-positive reals;

$$(3.35) \quad I_-(x) = \begin{cases} 0, & x \leq 0 \\ \infty, & x > 0 \end{cases}.$$

While (3.34) has no inequality constraints, the objective is not differentiable. As such, we must approximate the indicator function with a sequence of differentiable functions converging to it in the pointwise limit:

$$(3.36) \quad \tilde{I}_-(x) = -(1/t) \log(-x), \quad t > 0, \quad x \in \mathbb{R}_*^-.$$

Note, we use the convention that  $\tilde{I}_-(x > 0) = \infty$ . Observe that  $\tilde{I}_-(x) \rightarrow \infty$  as  $x \rightarrow 0^-$ . As  $t \rightarrow \infty$  with  $x < 0$ ,  $\tilde{I}_-(x) \rightarrow 0$ . The problem is now approximated with a differentiable objective:

$$(3.37) \quad \begin{aligned} & \min_{\mathbf{z} \in \mathbb{R}^{(d+1)N}} t f(\mathbf{z}) + \phi(\mathbf{z}) \\ & \text{subject to} \quad h_1(\mathbf{z}) = A\mathbf{z} - \mathbf{1} = 0, \end{aligned}$$

for  $t > 0$ , where the log barrier function  $\phi$  is given by

$$(3.38) \quad \phi(\mathbf{z}) = - \sum_{i=1}^{(d+2)N} \log(-f_i(\mathbf{z})).$$

We can solve (3.37) for increasing values of  $t$ , and use the solution from each previous value of  $t$  as the initialization for the problem with the next  $t$  value. This generates a sequence  $\mathbf{z}^*(t)$ , known as the *central path*, which converges to the optimal point  $\mathbf{z}^*$  as  $t \rightarrow \infty$  (if the problem is solvable). To each approximate problem, we can associated the modified KKT conditions, written in vector form as

$$(3.39) \quad \mathbf{r}_t(\mathbf{z}, \boldsymbol{\lambda}, \nu) = \begin{bmatrix} \nabla f(\mathbf{z}) + [\mathcal{D}(\mathbf{f}(\mathbf{z}))]^T \boldsymbol{\lambda} + A^T \nu \\ -\text{diag}(\boldsymbol{\lambda}) \mathbf{f}(\mathbf{z}) - (1/t) \mathbf{1}_{(d+2)N \times 1} \\ A\mathbf{z} - \mathbf{1} \end{bmatrix} = \begin{bmatrix} \mathbf{r}_{\text{dual}} \\ \mathbf{r}_{\text{cent}} \\ \mathbf{r}_{\text{pri}} \end{bmatrix} = 0.$$

The first block row is the *dual residual*. The second is the *centrality residual* (corresponding to the residual of the deformed KKT condition  $-\lambda_i f_i(\mathbf{z}) = 1/t$ ), and the last block row is the *primal residual*. Let

$$\mathbf{y} = (\mathbf{z}, \boldsymbol{\lambda}, \nu).$$

We can determine the Newton step for solving the KKT system (3.39) by finding its linearization about  $\mathbf{y}$ ;

$$(3.40) \quad \begin{aligned} \mathbf{r}_t(\mathbf{y} + \Delta \mathbf{y}) &\approx \mathbf{r}_t(\mathbf{y}) + \mathcal{D}(\mathbf{r}_t(\mathbf{y})) \Delta \mathbf{y} = 0 \\ \Rightarrow \Delta \mathbf{y} &= -\mathcal{D}(\mathbf{r}_t(\mathbf{y}))^{-1} \mathbf{r}_t(\mathbf{y}). \end{aligned}$$

Parsing this out, we have the following system to solve for the *primal dual* search direction  $\Delta \mathbf{y}$ :

$$(3.41) \quad \begin{bmatrix} \nabla^2 f(\mathbf{z}) + \boldsymbol{\lambda}^T \nabla^2 \mathbf{f}(\mathbf{z}) & [\mathcal{D}(\mathbf{f}(\mathbf{z}))]^T & A^T \\ -\text{diag}(\boldsymbol{\lambda}) \mathcal{D}(\mathbf{f}(\mathbf{z})) & -\text{diag}(\mathbf{f}(\mathbf{z})) & 0 \\ A & 0 & 0 \end{bmatrix} \begin{bmatrix} \Delta \mathbf{z} \\ \Delta \boldsymbol{\lambda} \\ \Delta \nu \end{bmatrix} = - \begin{bmatrix} \mathbf{r}_{\text{dual}} \\ \mathbf{r}_{\text{cent}} \\ \mathbf{r}_{\text{pri}} \end{bmatrix}.$$

The above system couples the primal and dual search directions through both the coefficient matrix and the residuals. In Section 3.2.3, we give formulae for the derivatives of the objective function (3.14). For clarity, we provide the relevant components here as well<sup>2</sup>. Recalling our partitioning of the solution variable  $\mathbf{z}$  defined in (3.16), we have

$$\begin{aligned}\mathcal{D}(f(\mathbf{z})) &= 2\mathbf{F}(\mathbf{z})^T \mathcal{D}(\mathbf{F}(\mathbf{z})) \in \mathbb{R}^{1 \times dN}, \\ \mathcal{D}^2(f(\mathbf{z})) &= 2 \left( \sum_{j=1}^{(d+2)N} F_j(\mathbf{z}) [\mathcal{D}^2(F_j(\mathbf{z}))] + \left( [\mathcal{D}(\mathbf{F}(\mathbf{z}))] \right)^T \mathcal{D}(\mathbf{F}(\mathbf{z})) \right) \in \mathbb{R}^{dN \times dN} \\ \mathcal{D}(\mathbf{f}(\mathbf{z})) &= G, \\ \mathcal{D}^2(\mathbf{f}(\mathbf{z})) &= 0.\end{aligned}$$

The components of  $\mathcal{D}(\mathbf{F}(\mathbf{z}))$  are given in (3.42), and those of  $\mathcal{D}^2(F_j(\mathbf{z}))$  are stated in (3.45)-(3.53).

**3.2.3. Derivatives of the objective functions.** We can use the machinery developed in Section 1.1 to compute derivatives of (3.14). Given that these tools are only at our disposal for  $d \leq 2$ , we proceed for now with  $d = 2$  and  $\mathbf{x} \in \mathcal{T}^2$ . We view each component of the objective as a function  $F_j : \mathbb{R}^{3N} \rightarrow \mathbb{R}$ , and note that  $\mathbf{P}$  is defined in the  $(a, b, c)$  Jacobi basis. Applying the derivative operators defined in (1.7), we have at a point  $(\mathbf{X}, \mathbf{Y}, \mathbf{W})$ ,

$$\begin{aligned}F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W}) &= \mathbf{e}_j^T \mathbf{P}(\mathbf{X}, \mathbf{Y}) \mathbf{W} - \delta_{1j} = \sum_{i=1}^N P_j(x_i, y_i) w_i - \delta_{1j}, \\ \frac{\partial F_j}{\partial x_k} &= w_k \mathbf{P}^x(x_k, y_k)^T \mathcal{D}_x \mathbf{e}_j, \quad \frac{\partial F_j}{\partial y_k} = w_k \mathbf{P}^y(x_k, y_k)^T \mathcal{D}_y \mathbf{e}_j, \quad \frac{\partial F_j}{\partial w_k} = \mathbf{P}(x_k, y_k)^T \mathbf{e}_j, \\ \frac{\partial^2 F_j}{\partial x_l \partial x_k} &= w_k \mathbf{P}^{xx}(x_k, y_k)^T \mathcal{D}_{xx} \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial x_l \partial y_k} = w_k \mathbf{P}^{xy}(x_k, y_k)^T \mathcal{D}_{xy} \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial x_l \partial w_k} = \mathbf{P}^x(x_k, y_k)^T \mathcal{D}_x \mathbf{e}_j \delta_{lk}, \\ \frac{\partial^2 F_j}{\partial y_l \partial x_k} &= w_k \mathbf{P}^{yx}(x_k, y_k)^T \mathcal{D}_{yx} \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial y_l \partial y_k} = w_k \mathbf{P}^{yy}(x_k, y_k)^T \mathcal{D}_{yy} \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial y_l \partial w_k} = \mathbf{P}^y(x_k, y_k)^T \mathcal{D}_y \mathbf{e}_j \delta_{lk}, \\ \frac{\partial^2 F_j}{\partial w_l \partial x_k} &= \mathbf{P}^x(x_k, y_k)^T \mathcal{D}_x \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial w_l \partial y_k} = \mathbf{P}^y(x_k, y_k)^T \mathcal{D}_y \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial w_l \partial w_k} = 0 \cdot \delta_{lk},\end{aligned}$$

where

$$\begin{aligned}\mathcal{D}_x &= \mathcal{D}_{x, (a, b, c)}^{a+1, b, c+1}, \quad \mathbf{P}^x = \mathbf{P}^{(a+1, b, c+1)}, \quad \mathcal{D}_y = \mathcal{D}_{x, (a, b, c)}^{a, b+1, c+1}, \quad \mathbf{P}^y = \mathbf{P}^{(a, b+1, c+1)} \\ \mathcal{D}_{xx} &= \mathcal{D}_{x, (a+1, b, c+1)}^{(a+2, b, c+2)} \mathcal{D}_{x, (a, b, c)}^{(a+1, b, c+1)}, \quad \mathbf{P}^{xx} = \mathbf{P}^{(a+2, b, c+2)}, \\ \mathcal{D}_{yy} &= \mathcal{D}_{y, (a, b+1, c+1)}^{(a, b+2, c+2)} \mathcal{D}_{y, (a, b, c)}^{(a, b+1, c+1)}, \quad \mathbf{P}^{yy} = \mathbf{P}^{(a, b+2, c+2)} \\ \mathcal{D}_{xy} &= \mathcal{D}_{x, (a, b+1, c+1)}^{(a+1, b+1, c+2)} \mathcal{D}_{y, (a, b, c)}^{(a, b+1, c+1)}, \quad \mathbf{P}^{xy} = \mathbf{P}^{(a+1, b+1, c+2)}, \\ \mathcal{D}_{yx} &= \mathcal{D}_{y, (a+1, b, c+1)}^{(a+1, b+1, c+2)} \mathcal{D}_{x, (a, b, c)}^{(a+1, b, c+1)}, \quad \mathbf{P}^{yx} = \mathbf{P}^{xy}\end{aligned}$$

<sup>2</sup>For a function  $\mathbf{u} : \mathbb{R}^N \rightarrow \mathbb{R}^M$ , we note the distinction between  $\mathcal{D}(\mathbf{u}(\mathbf{z}))$  and  $\nabla(\mathbf{u}(\mathbf{z}))$ . The former is an element of  $L(\mathbb{R}^N, \mathbb{R}^M)$ , equivalently viewed as a set of  $M$  elements of  $L(\mathbb{R}^N, \mathbb{R})$ , or  $M$  row vectors in  $\mathbb{R}^N$  (left argument of dot product). The latter evaluated at a point is as an element of  $L(\mathbb{R}^M, \mathbb{R}^N)$ , equivalently viewed as a set of  $M$  elements of  $L(\mathbb{R}, \mathbb{R}^N)$ , or  $M$  column vectors in  $\mathbb{R}^N$  (right argument of dot product). TLDR:  $[\mathcal{D}(\mathbf{u})]^T = \nabla(\mathbf{u})$

Now, reverting to the view of the objective as  $\mathbf{F} : \mathbb{R}^{3N} \rightarrow \mathbb{R}^M$ , we can write

$$(3.42) \quad [\mathcal{D}(\mathbf{F})]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} = [\mathcal{D}_x^T \mathbf{P}^x(\mathbf{X}, \mathbf{Y}) \odot \mathbf{W} \quad \mathcal{D}_y^T \mathbf{P}^y(\mathbf{X}, \mathbf{Y}) \odot \mathbf{W} \quad \mathbf{P}(\mathbf{X}, \mathbf{Y})] \in \mathbb{R}^{M \times 3N},$$

and we can use this formula in the Gauss-Newton update (3.17). If we instead consider the equivalent problem of finding zeros of the objective norm  $f : \mathbb{R}^{3N} \rightarrow \mathbb{R}$ ,  $\mathbf{f}(\mathbf{X}, \mathbf{Y}, \mathbf{W}) = \|\mathbf{F}\|_2^2$ , (solved by the iteration (3.20)), we must chain out derivatives to find the mapping  $\mathcal{D}(\mathbf{f}) : \mathbb{R}^{3N} \rightarrow L(\mathbb{R}^{3N}, \mathbb{R})$  at a point  $(\mathbf{X}, \mathbf{Y}, \mathbf{W})$  is given by

$$(3.43) \quad [\mathcal{D}(f)]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} = 2\mathbf{F}(\mathbf{X}, \mathbf{Y}, \mathbf{W})^T [\mathcal{D}(\mathbf{F})]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} \in \mathbb{R}^{1 \times 3N},$$

where  $L(U, V)$  is the space of linear mappings between finite dimensional vector spaces  $U, V$ . In this case, the Hessian matrix of  $\mathbf{f}$  is a mapping  $\mathcal{D}^2(\mathbf{f}) : \mathbb{R}^{3N} \rightarrow L(\mathbb{R}^{3N}, \mathbb{R}^{3N})$  given at a point  $(\mathbf{X}, \mathbf{Y}, \mathbf{W})$  by

$$(3.44) \quad \begin{aligned} [\mathcal{D}^2(f)]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} &= 2 \sum_{j=1}^M F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W}) [\mathcal{D}^2(F_j)]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} \\ &\quad + 2 \left( [\mathcal{D}(\mathbf{F})]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} \right)^T [\mathcal{D}(\mathbf{F})]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} \\ &\in \mathbb{R}^{3N \times 3N} \end{aligned}$$

The vector Hessian of  $\mathbf{F}$  appears in (3.44), and we can make sense of its tensor structure by reviewing the mixed partial formulae given at the beginning of this section. In particular, we have

$$(3.45) \quad \partial_{\mathbf{X}\mathbf{X}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left( \mathbf{1}_N^T \otimes \left( \mathbf{W} \odot \mathbf{P}^{xx}(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_{xx} \mathbf{e}_j \right) \right),$$

$$(3.46) \quad \partial_{\mathbf{Y}\mathbf{X}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left( \mathbf{1}_N^T \otimes \left( \mathbf{W} \odot \mathbf{P}^{yx}(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_{yx} \mathbf{e}_j \right) \right),$$

$$(3.47) \quad \partial_{\mathbf{W}\mathbf{X}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left( \mathbf{1}_N^T \otimes \left( \mathbf{P}^x(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_x \mathbf{e}_j \right) \right),$$

$$(3.48) \quad \partial_{\mathbf{X}\mathbf{Y}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left( \mathbf{1}_N^T \otimes \left( \mathbf{W} \odot \mathbf{P}^{xy}(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_{xy} \mathbf{e}_j \right) \right),$$

$$(3.49) \quad \partial_{\mathbf{Y}\mathbf{Y}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left( \mathbf{1}_N^T \otimes \left( \mathbf{W} \odot \mathbf{P}^{yy}(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_{yy} \mathbf{e}_j \right) \right),$$

$$(3.50) \quad \partial_{\mathbf{W}\mathbf{Y}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left( \mathbf{1}_N^T \otimes \left( \mathbf{P}^y(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_y \mathbf{e}_j \right) \right),$$

$$(3.51) \quad \partial_{\mathbf{X}\mathbf{W}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left( \mathbf{1}_N^T \otimes \left( \mathbf{P}^x(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_x \mathbf{e}_j \right) \right),$$

$$(3.52) \quad \partial_{\mathbf{Y}\mathbf{W}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left( \mathbf{1}_N^T \otimes \left( \mathbf{P}^y(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_y \mathbf{e}_j \right) \right),$$

$$(3.53) \quad \partial_{\mathbf{W}\mathbf{W}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = 0_{N \times N},$$

where  $\cdot \otimes \cdot$  symbolizes the Kronecker product, and  $\cdot \odot \cdot$  is component-wise multiplication along the columns of the argument to the right<sup>3</sup>. We verify the correctness of our formulae for the first and second derivatives of our vector and scalar objectives by comparing with second-order finite difference approximations. In particular, we show in Figure 2 the relative error in the 2-norm between the analytical evaluation and the finite difference approximation. Since the difference scheme is second order, we expect the error to behave like  $\mathcal{O}(h^2)$ , where  $h$  is the stencil width. That is, we should observe that

$$\log(\|\mathcal{D}(\mathbf{F}) - \hat{\mathcal{D}}(\mathbf{F})\|) \approx 2 \log(h) + \beta,$$

where  $\beta$  is the order constant, and  $\hat{\mathcal{D}}(\mathbf{F})$  is the finite difference approximation. The same relationship holds for  $\mathcal{D}^2(\mathbf{F})$  and other derivatives as well. On a log – log scale, this means the slope of the error lines should be around 2. Indeed, we do observe this relationship in the regime of asymptotic convergence, before round-off error dominates truncation error for the finite difference scheme.

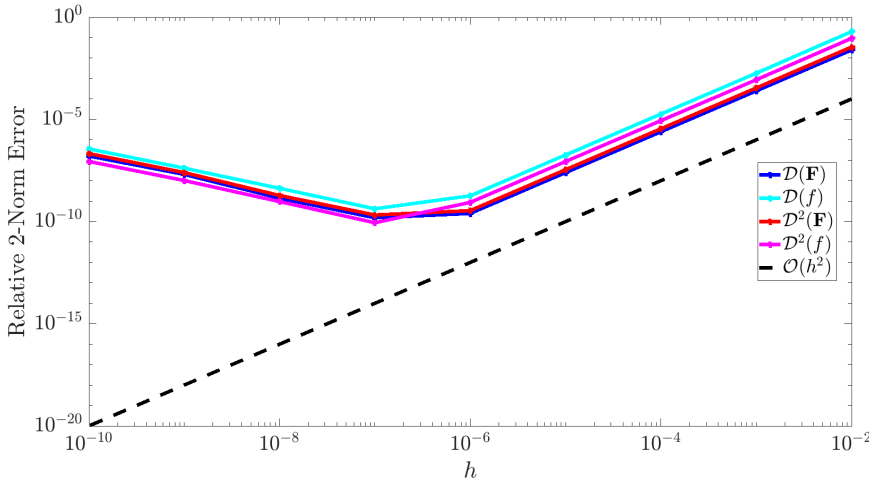


FIG. 2. Relative error in the 2-norm (Frobenius for matrix valued derivatives) of the analytical derivative formulae when compared to second order finite difference approximations. We see that our formulae must be correct, if our finite difference implementation is correct (which is an initial assumption). The kink in the error curves after which the error increases is typical of a finite difference scheme, and occurs at the “usual” location for at least twice differentiable functions, i.e.  $h \approx 1e-6$ . This corresponds to the point where numerical round-off error (due to IEEE floating point specification) dominates truncation error, and can be explicitly determined via error analysis by making reference to the Taylor error term in the difference scheme, with propagation of  $\varepsilon_{mach}$ .

The plot also illustrates the utility of having available to us the analytical derivative formulae. In particular, we note that the solution variables to any of the proposed iterations involve nodes interior to  $\mathcal{T}^2$ , and the zero of the objective function is only some  $\varepsilon$  perturbation away from the initial points obtained by JEVD or complex eigenvalue decomposition. If this  $\varepsilon$  is smaller than  $1e-6$  for any component of

<sup>3</sup>Viewing  $\mathbf{W}$  as a column of size  $N$ , we compute the component-wise product with columns of the matrix on the right. It would be more correct to define a function `repmat(W, k)`, which takes a vector  $W \in \mathbb{R}^N$  and repeats it  $k$  times, producing a matrix in  $\mathbb{R}^{N \times k}$  with each of the  $k$  columns a copy of  $W$ . However, for  $N \neq M$ , the ambiguity in matching sizes of the arguments to  $\odot$  is eliminated.

the solution, which I’ve empirically observed is the case, finite difference schemes are not appropriate for approximating the vector Hessian or gradient of the objective.

*Remark 3.2.* For this sort of simple “reverse” verification test, we assume that we don’t know if our analytical formulae are correct, and that a *verified* finite difference implementation is available. Then, we are essentially re-verifying the finite difference implementation, which on success, certifies correctness of the analytical formulae.

TODO: Derive the derivative and promotion operators for the general case of  $d > 2$ .